Jenkins CI/CD Challenge

Expanded Practical Summary with Screenshots

Application: Hiring App

Repository: betawins/hiring-app

Branch: main

The document also records an initial Nexus authentication failure (401 Unauthorized). The credentials were then independently verified and Maven was configured with a settings.xml file that received the Jenkins-injected credentials. After this change, deployment completed successfully.

Common Build and Test Commands

Maven build:

```
mvn clean package -DskipTests
```

This cleans previous build output, packages the application, and skips test execution during the package phase. The resulting artifact is target/hiring.war.

Test:

```
mvn test
```

The test command completed successfully. The source project had no executable tests, so the recorded output was No tests to run.

Declarative Pipeline

Job: Hiring-App-Declarative-Pipeline-Challenge

The Declarative implementation uses the standard Jenkins structure: pipeline → agent → stages → steps. Its purpose is to provide a clean and predictable CI/CD workflow.

Execution flow

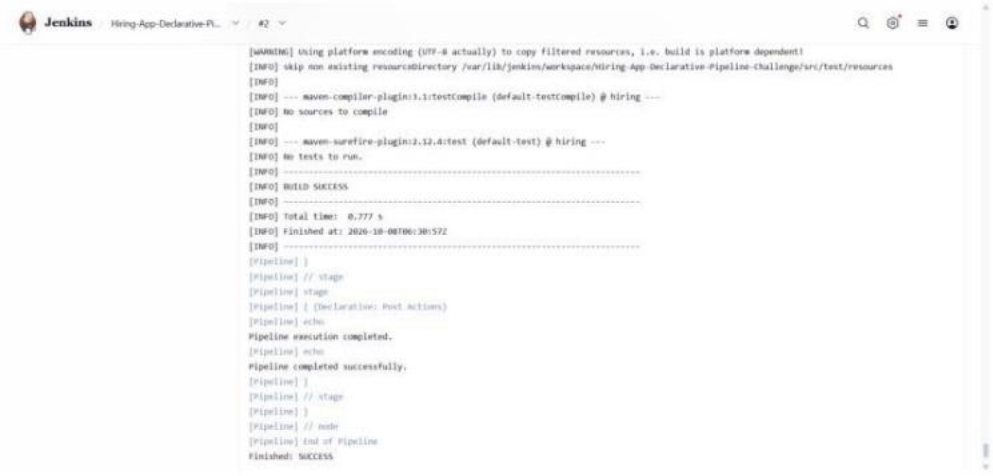
Git Clone → Maven Build → Test → SonarQube Analysis → Nexus Artifact

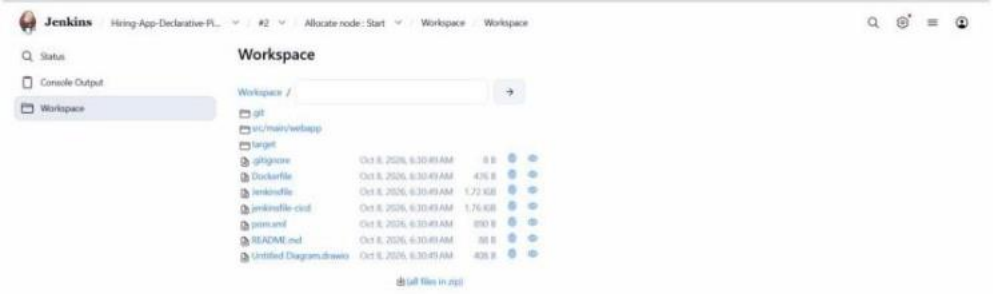
Key implementation points

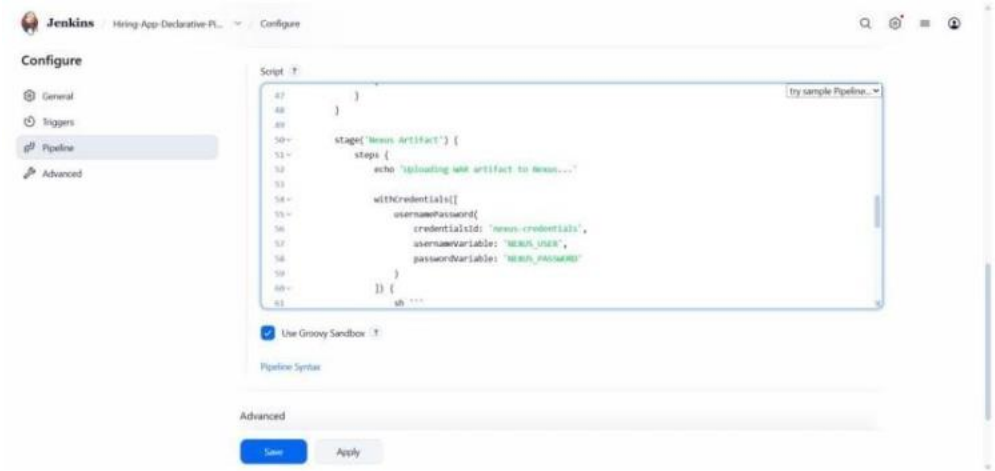
- **Git checks out the main branch from the Hiring App repository.**
- **Maven packages the application into target/hiring.war.**
- **Maven test is executed as a separate stage.**
- **SonarQube analysis uses the Jenkins-managed sonarqube-token credential.**
- **Nexus upload uses nexus-credentials and Maven settings.xml.**
- **The pipeline has post actions for success, failure, and always-run completion messaging.**

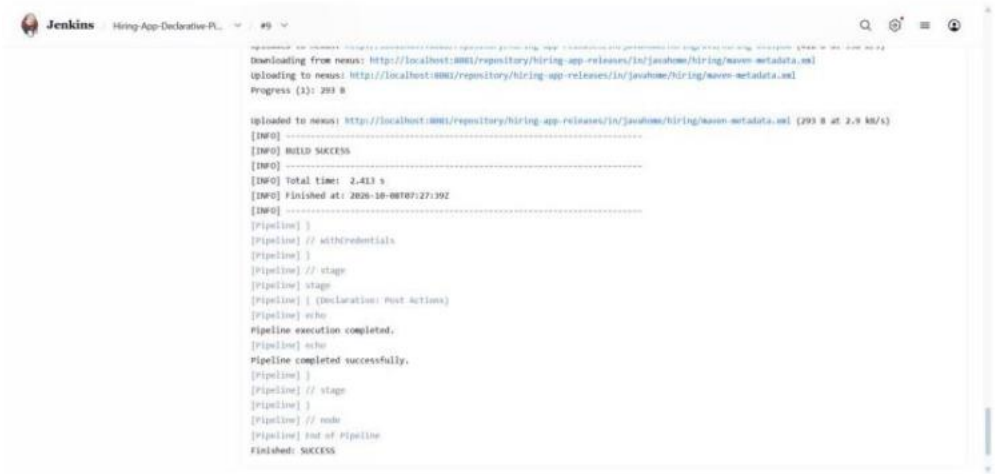
Representative Jenkins screenshots













Result: Git Clone, Maven Build, Test, SonarQube Analysis and Nexus Artifact all completed successfully; the final Jenkins result was Finished: SUCCESS.

Scripted Pipeline

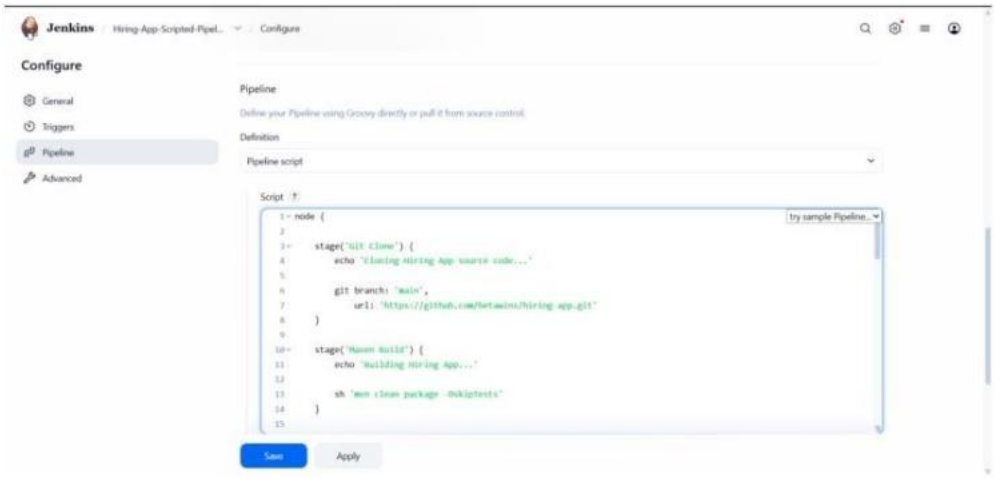
Job: Hiring-App-Scripted-Pipeline-Challenge

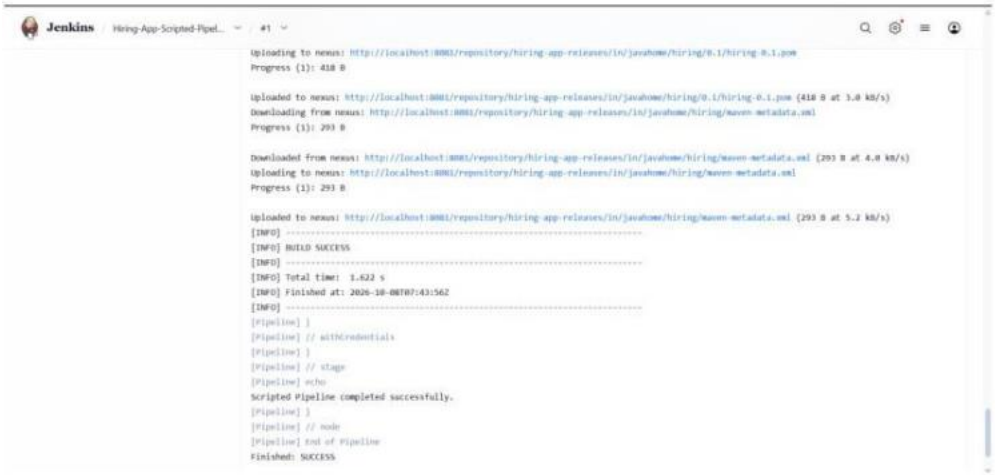
The Scripted implementation uses a Groovy-based structure beginning with node { }. Stages are placed inside the node block, giving more flexibility than the predefined Declarative structure.

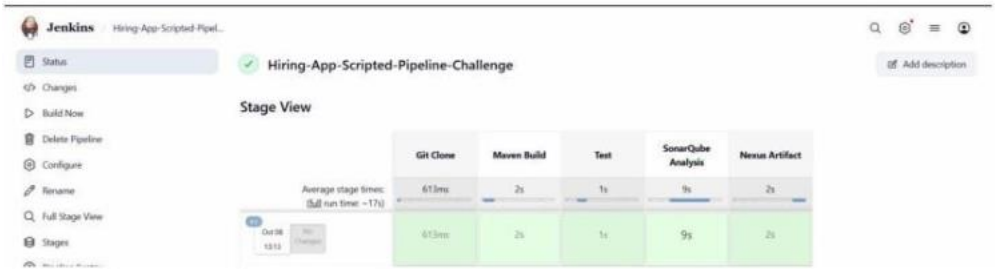
Structure

node { → stage('Git Clone') → stage('Maven Build') → stage('Test') → stage('SonarQube Analysis') → stage('Nexus Artifact') }

- **The Git stage explicitly specifies the main branch because the repository uses main.**
- **The Maven build generates the WAR file.**
- **The Test stage runs mvn test.**
- **SonarQube authentication is handled through withCredentials.**
- **Nexus authentication is handled through username/password credentials injected into Maven settings.xml.**
- **The source document notes that the code was pasted directly into Jenkins' Pipeline script section.**







Result: The Scripted Pipeline completed all five CI/CD activities successfully and ended with Finished: SUCCESS.

Multi-Stage Pipeline

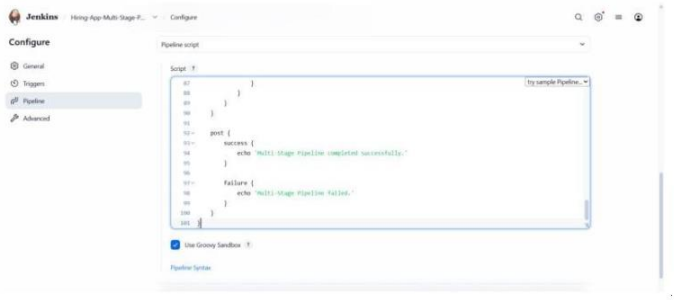
Job: Hiring-App-Multi-Stage-Pipeline-Challenge

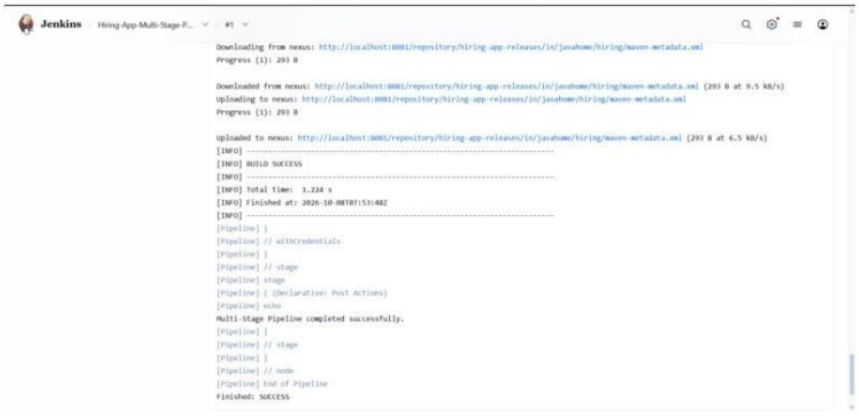
The Multi-Stage implementation emphasizes organization. Each major activity is represented as a separate stage, so Jenkins can clearly show where the pipeline is currently executing and where a failure occurred.

Stages

- **Git Clone**
- **Maven Build**
- **Test**
- **SonarQube Analysis**
- **Nexus Artifact**

The stages run sequentially. If an earlier stage fails, later stages normally do not proceed. This separation makes the pipeline easier to understand, monitor, and troubleshoot.







The application was cloned, built, tested, analyzed, and stored in Nexus, ending with Finished: SUCCESS.

Parallel Stage Pipeline

Job: Hiring-App-Parallel-Stage-Pipeline-Challenge

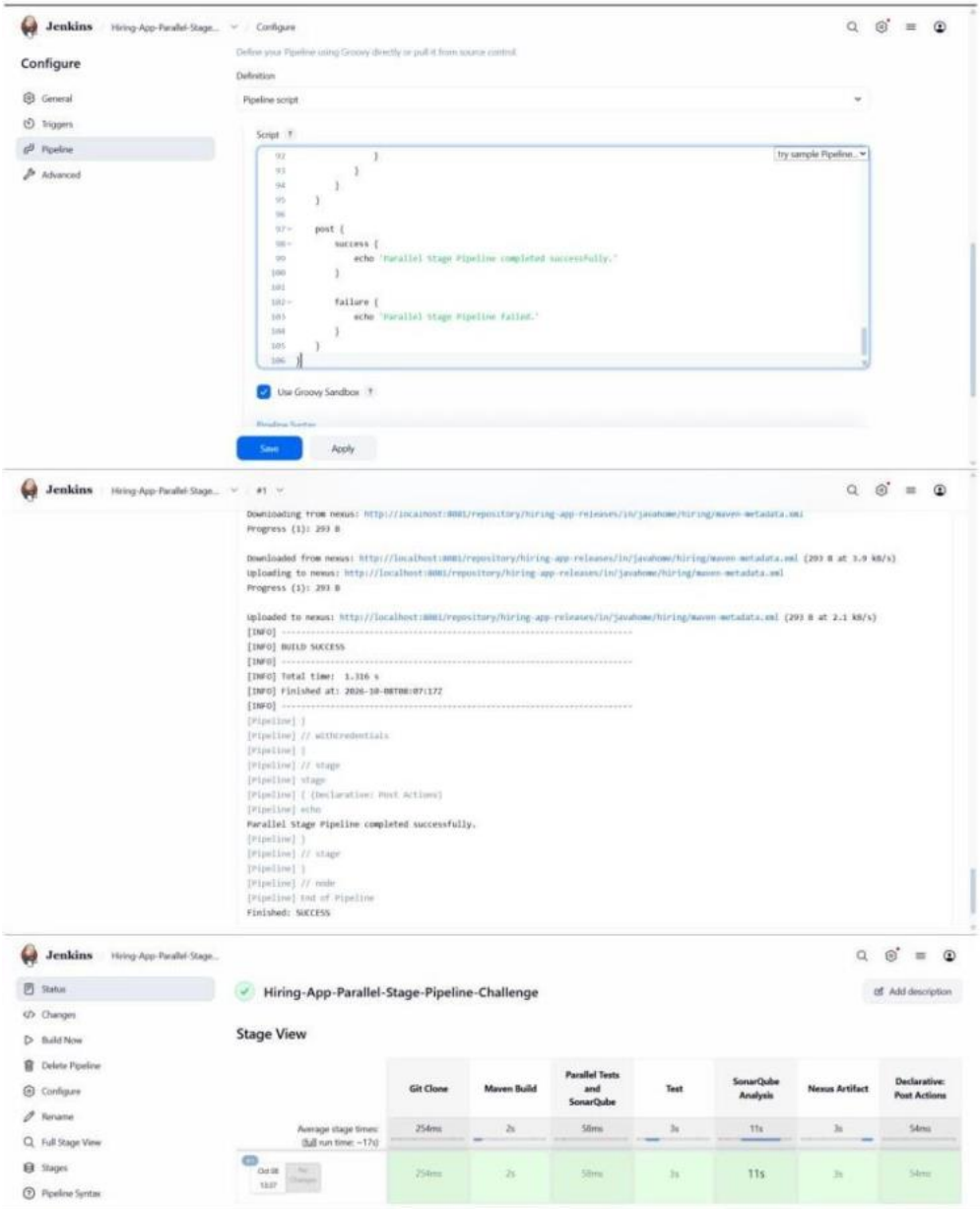
The Parallel Stage implementation changes the execution model after Maven Build. Test and SonarQube Analysis are independent in this challenge, so they are placed inside a Jenkins parallel block.

Flow

Git Clone → Maven Build → [Test || SonarQube Analysis] → Nexus Artifact

- **Git Clone runs first.**
- **Maven Build runs next.**
- **Test and SonarQube Analysis start as parallel branches.**
- **Jenkins waits until both parallel branches finish.**
- **Nexus Artifact upload runs after the parallel block completes.**

Jenkins waits for the parallel branches to finish before moving to the next stage, which in this task was **Nexus Artifact**.



The original document explains the performance benefit with an example: if Test takes 2 minutes and SonarQube takes 3 minutes, sequential execution is approximately 5 minutes, while parallel execution can be approximately 3 minutes. Actual time depends on Jenkins resources and workload.

Result: The parallel branches and the final Nexus stage completed successfully; the final result was Finished: SUCCESS.

Nexus Artifact Deployment - Detailed View

All four pipeline implementations publish the generated WAR using Maven's deploy:deploy-file operation. The important artifact information recorded in the source is:

Property	Value
Group ID	in.javahome
Artifact ID	hiring
Version	0.1
Packaging	war
File	target/hiring.war
Repository	hiring-app-releases

Deployment command used by the documented implementations:

```
mvn deploy:deploy-file \ -s
/var/lib/jenkins/.m2/settings.xml \
-DgroupId=in.javahome \
-DartifactId=hiring \
-Dversion=0.1 \
-Dpackaging=war \
-Dfile=target/hiring.war \
-DrepositoryId=nexus \
-Durl=http://localhost:8081/repository/hiring-app-releases/ \
-DgeneratePom=true
```

The successful deployment generated hiring-0.1.war, hiring-0.1.pom, and maven-metadata.xml. The document records the Maven result as BUILD SUCCESS.

SonarQube Integration - Detailed View

The SonarQube stage performs static code analysis using the Maven SonarQube scanner. The SonarQube token is stored in Jenkins Credentials and exposed temporarily to the shell as SONAR_TOKEN.

Credential pattern

```
withCredentials([ string(credentialsId: 'sonarqube-token', variable: 'SONAR_TOKEN') ])
```

Scanner command documented in the source:

```
mvn org.sonarsource.scanner.maven:sonar-maven-
plugin:5.8.0.7211:sonar \ - Dsonar.host.url=http://localhost:9000 \

-Dsonar.token=$SONAR_TOKEN
```

The source records ANALYSIS SUCCESSFUL and BUILD SUCCESS, confirming that the project was analyzed and the report was uploaded.

Security point: the token is referenced through Jenkins Credentials instead of placing the secret directly in the pipeline script.

Comparing the Four Pipeline Styles

The original document's comparison table highlights the main distinction between the four approaches. The table below restates those points in a concise form.

Pipeline	Structure	Main strength	Example from challenge
Declarative	pipeline → agent → stages → steps	Simple, structured CI/CD	Standard Hiring App flow
Scripted	node → stage → commands	Flexibility and Groovy logic	Scripted Hiring App flow
Multi-Stage	Multiple logical stage blocks	Clear organization	Git → Build → Test → Sonar →
Parallel	parallel { ... }	Concurrent execution	Test + SonarQube together

Important Jenkins Concepts to Remember

node - Allocates a Jenkins executor/workspace where a Scripted Pipeline runs.

Stage - Groups related operations and makes the pipeline easier to understand

in Jenkins. sh - Runs a shell command on the Jenkins Linux machine.

git - Checks out the required repository and branch. With Credentials -

Temporarily provides Jenkins-managed secrets to pipeline steps. parallel - Runs

independent

stages concurrently and waits for them before moving on.

Final Practical Summary

The complete challenge demonstrates how the same application can be delivered through different Jenkins pipeline designs. The core delivery process remains the same, while the pipeline structure changes according to the requirement.

- **Declarative: use when you want a standard, readable Jenkins structure.**
- **Scripted: use when you need more Groovy flexibility or dynamic logic.**
- **Multi-Stage: use stages to make each CI/CD responsibility visible and easy to troubleshoot.**
- **Parallel: use concurrency when stages are independent and can safely run at the same time.**

Final flow to remember:

GitHub → Git Clone → Maven Build → Test → SonarQube → Nexus

Parallel version: GitHub → Git Clone → Maven Build → [Test || SonarQube] → Nexus